

CSMP++

Basic Source Code Documentation Guidelines

CSMP++ Source Code – Basic Doxygen
Commands

Documenting Classes

Brief description of Class/Struct/Enum

```
#ifndef CSMP_INDEX_H
#define CSMP_INDEX_H

#include "CSMP_definitions.h"

namespace csmp {

    /// Accessor for discretized physical variables

    struct Index {
        Index();
        Index( VARIABLE_TYPE ty, PLACEMENT pl, size_t idx );
        Index( const csmp::Index& idx );

        Index& operator=( const csmp::Index& idx );
        bool operator==( const csmp::Index& i ) const;
        bool operator!=( const csmp::Index& i ) const;
        bool operator<( const csmp::Index& i ) const; // for stl
        bool IsDefined() const;

        void Out() const;

        VARIABLE_TYPE type;
        PLACEMENT place;
        size_t index;
    };

    // type parsing (although slow)
    template<typename Var> VARIABLE_TYPE variableType( const Var& );

    // console input and output
    std::ostream& operator<<( std::ostream& stream, const csmp::Index& o );
    std::istream& operator>>( std::istream& stream, csmp::Index& o );

} // csmp
```

Detailed description of Class/Struct/Enum

Not above class declaration
but in the csmc namespace.
Preferably at bottom of
header (.h) file.

@class / @struct / @enum
<classname>
<filename>
<location>

@section
<internReferenceName>
<title>

@code
<codeSnippet>
@endcode

```
namespace csmc {  
    /**  
    @struct Index Index.h "main_library/Index.h" ←  
    @author S.K. Matthaei ←  
    @author S. Geiger  
    @author Stephen G. Roberts  
    @section motivation Motivation ←  
    Class encapsulates the three values that are required to retrieve a  
    CSP variable from a model based on an unstructured mesh: The Type()  
    distinguishes scalars, vectors and tensors from one-another since these  
    have different storage requirements.  
    The Place() specifies whether a variable is discretized on the nodes,  
    elements, faces or on constraint points. This has implications as to  
    how the variable is interpolated over the model domain.  
    The Index() is used internally by CSMP for housekeeping. It corresponds  
    to the number that the variable has in an alphabetically ordered  
    map of variables of the same type and placement, in the same model.  
    @section design Design Intent  
    Create a universal accessor for discretized physical variables in a  
    model.  
    ...  
    @section examples Application Examples  
    To get an Index for a variable of interest in order to perform  
    a computation, do the following:  
    @code ←  
    Index variable_key = prop_database.StorageKey("variable");  
    @endcode  
*/
```

Documenting Functions

brief description

detailed description

```
#include "Index.h"
#include "TensorVariable.h"
#include "CSMP_highLevelUtilities.h"

using namespace std;

namespace csmp {
    /** comparator less
    Provides required functionality for use in an STL container.

    @test Tested OK
    @attention Do not use this as an example for how to code
    since it deduces information at runtime which should already be
    known at compile time.
    @param i a csmp::Index
    @return true if smaller than argument
    */
    bool Index::operator<( const csmp::Index& i ) const // for stl
    {
        const size_t lhs((type+1) * (place+1) * (index+1U));
        const size_t rhs((i.type+1) * (i.place+1) * (i.index+1U));
        return (lhs < rhs);
    }
}
```

@test

@attention

@param <parameter>

@return

@bug

@todo

@section...

Documenting Members

```
private:
    const double swi_,   initial saturation in matrix
    swc_,    ///< connate water saturation
    snr_,    ///< residual non-wetting phase saturation
    lambda_, ///< Brooks-Corey parameter (water-wet system)
    pd_,     ///< capillary entry pressure
    km_,     ///< matrix permeability
    phi_,    ///< matrix porosity
    muw_,    ///< viscosity water
    mun_;    ///< viscosity non-wetting phase
};
```

CSMP++ Source Code – Porting Documentation

```

/**
void Mesh<dim>::OutputMeshTo( VSet& vset ) const

\n \b Description
Writes current mesh connectivity information to the supplied VSet.
Note that no property data are output by this method.

\n \b Input \b Arguments
A VSet preferably empty, which is resized first, if necessary and
into which the mesh connectivity information is input.

\n \b Implementation
Typically, for a CSMP computation uninitialized data default to NAN. Since
it sometimes makes sense, however, to use a 2D mesh as an input into
a three-dimensional model, the z-coordinate is initialized to zero if the
model is not three dimensional.

\n \b Application
A mesh is output to a VSet, for instance, when the whole model is
to be saved to a binary file.

\n \b Messages
If the VSet is initialized successfully the method will report this.

tested: O.K. */
template<size_t dim>
void Mesh<dim>::OutputMeshTo( const FiniteElementManager& fem_manager,
                             VSet<dim>& vset ) const
{
    assert( !adaptive_remeshing_ );

```

```

/** Writes Mesh to VSet

```

Writes current mesh connectivity information to the supplied VSet. Note that no property data are output by this method.

@param fem_manager The relevant Element Manager
 @param vset A VSet preferably empty, which is resized first, if necessary and into which the mesh connectivity information is input.

@section implementation Implementation
 Typically, for a CSMP computation uninitialized data default to NAN. Since it sometimes makes sense, however, to use a 2D mesh as an input into a three-dimensional model, the z-coordinate is initialized to zero if the model is not three dimensional.

@section application Application
 A mesh is output to a VSet, for instance, when the whole model is to be saved to a binary file.

@attention If the VSet is initialized successfully the method will report this.

@test Tested OK

```

*/
template<size_t dim>
void Mesh<dim>::OutputMeshTo( const FiniteElementManager& fem_manager,
                             VSet<dim>& vset ) const
{
    assert( !adaptive_remeshing_ );

```



CSMP++ Source Code – Sections

CLASSES

brief description

detailed description

@section motivation Motivation

@section design Design Intent

@section applicability Applicability

@section collaborations Collaborations

@section implementation Implementation

@section examples Application Examples

see <main_library/Index.h>

FUNCTIONS

brief description

detailed description

@section implementation Implementation

@section application Application

see <main_library/Mesh.cpp>

Doxygen Documentation

<file:///C:/Documents%20and%20Settings/Owner/Desktop/CSMPdoxygen/Out/html/index.html>